Day 29

Introduction to Promises:

What is a Promise?

A Promise in JavaScript is an object that represents a value that will be available now, later, or never. It's used to handle asynchronous operations (things that take time).

A Promise can be in 3 states:

- **pending** → still running
- **fulfilled** → completed successfully
- **rejected** → failed

Other Important Details About Promises

- A Promise represents something that takes time, like fetching data from a server.
- .then() handles successful results.
- .catch() handles errors.
- .finally() runs no matter what (optional).
- Promises help avoid callback hell.
- Very commonly used with fetch() and async/await.

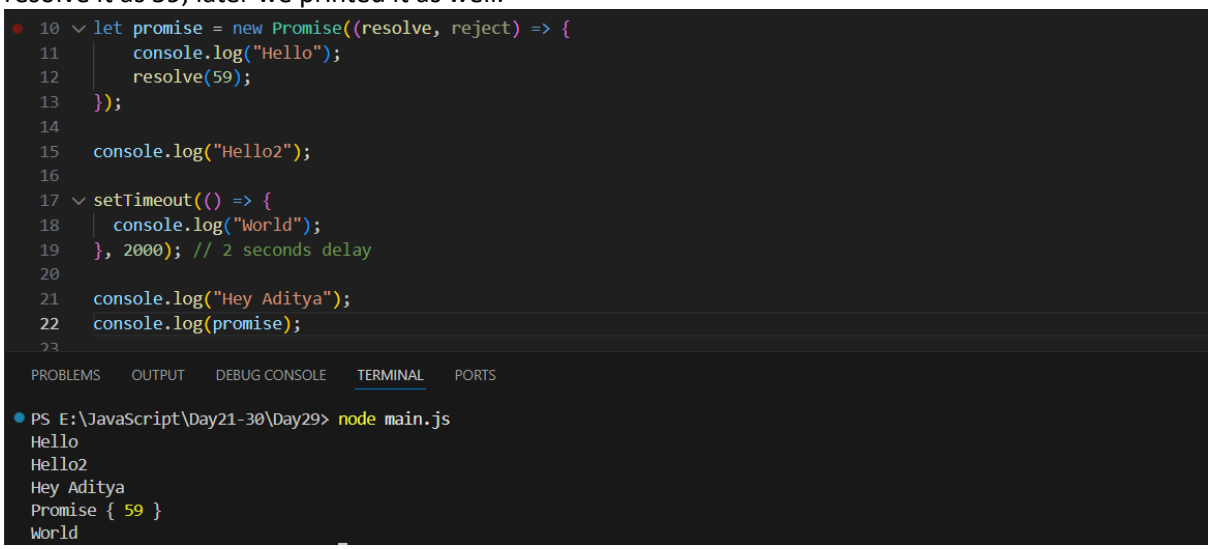
Syntax of promise:

```
let promise = new Promise(function(resolve, reject) {
  // Do some work...
  // If successful:
  resolve(value);
  // If an error happens:
  reject(error);
});
```

Example: a basic example where the "World" will be printed 2 seconds after everything else is printed.

```
JS main.js > ...
1   console.log("Hello");
2
3   setTimeout(() => {
4     console.log("World");
5   }, 2000); // 2 seconds delay
6
7   console.log("Hey Aditya");
8

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS E:\JavaScript\Day21-30\Day29> node main.js
Hello
Hey Aditya
World
```

Example: introducing promise in the code, basically we are saying that if the promise is fulfilled then resolve it as 59, later we printed it as well.



```
let promise = new Promise((resolve, reject) => {
    console.log("Hello");
    resolve(59);
});

console.log("Hello2");

setTimeout(() => {
  console.log("World");
}, 2000); // 2 seconds delay

console.log("Hey Aditya");
console.log(promise);
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS E:\JavaScript\Day21-30\Day29> node main.js
Hello
Hello2
Hey Aditya
Promise { 59 }
World
```

When the code runs, the Promise is created and the function inside it runs immediately, so the first thing printed is “Hello.” The Promise is then resolved with the value 59. After that, the next line prints “Hello2.” The setTimeout schedules the message “World” to be printed after 2 seconds, but it does not run immediately. The script continues, printing “Hey Aditya,” and then it prints the Promise object itself, which shows Promise { 59 } because it was already resolved. Finally, after a 2-second delay, the scheduled message “World” is printed.

Promise .then() and .catch():

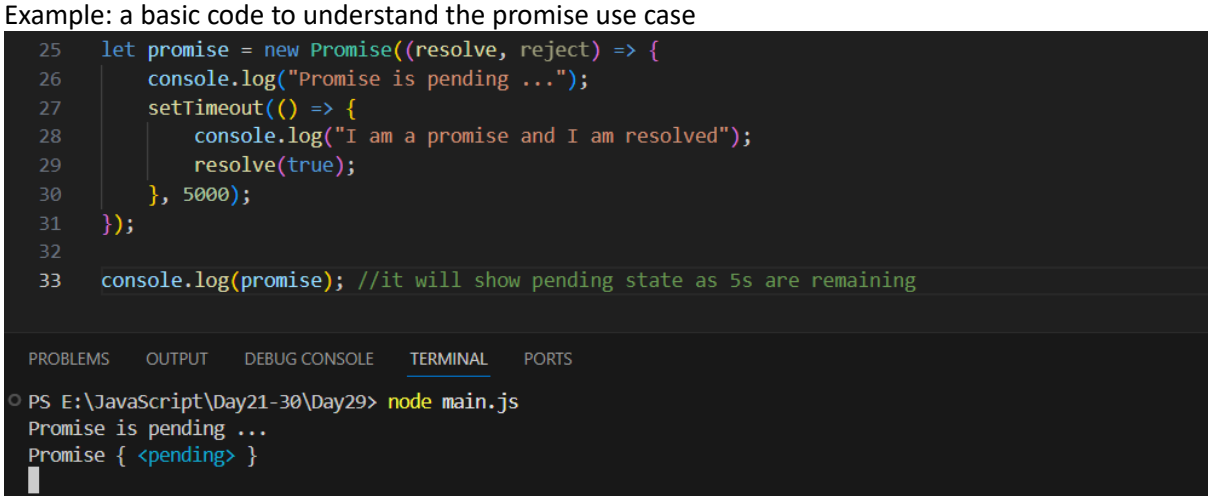
What is .then()?

.then() is used to run some code when a Promise is successfully resolved.The value passed to resolve() becomes the value received by .then().

What is .catch()?

.catch() is used to run some code when a Promise is rejected (when something goes wrong). The value passed to reject() becomes the error received by .catch().

Example: a basic code to understand the promise use case



```
let promise = new Promise((resolve, reject) => {
    console.log("Promise is pending ...");
    setTimeout(() => {
        console.log("I am a promise and I am resolved");
        resolve(true);
    }, 5000);
});

console.log(promise); //it will show pending state as 5s are remaining
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS E:\JavaScript\Day21-30\Day29> node main.js
Promise is pending ...
Promise { <pending> }
```

After 5 seconds:

```
PS E:\JavaScript\Day21-30\Day29> node main.js
Promise is pending ...
Promise { <pending> }
I am a promise and I am resolved
PS E:\JavaScript\Day21-30\Day29>
```

When the code starts running, a new Promise is created, and the message “Promise is pending ...” is printed immediately because the code inside a Promise runs right away. The setTimeout schedules the promise to resolve after 5 seconds, but it does not run yet. Then the line console.log(promise) is executed, and since the 5 seconds have not passed, it prints the Promise in its pending state: Promise { <pending> }. After 5 seconds, the function inside setTimeout runs, printing “I am a promise and I am resolved”, and the Promise finally becomes resolved with the value true.

Example: a promise which has been fulfilled and rejected. (Note: Fulfilled then it can either be resolved or rejected)

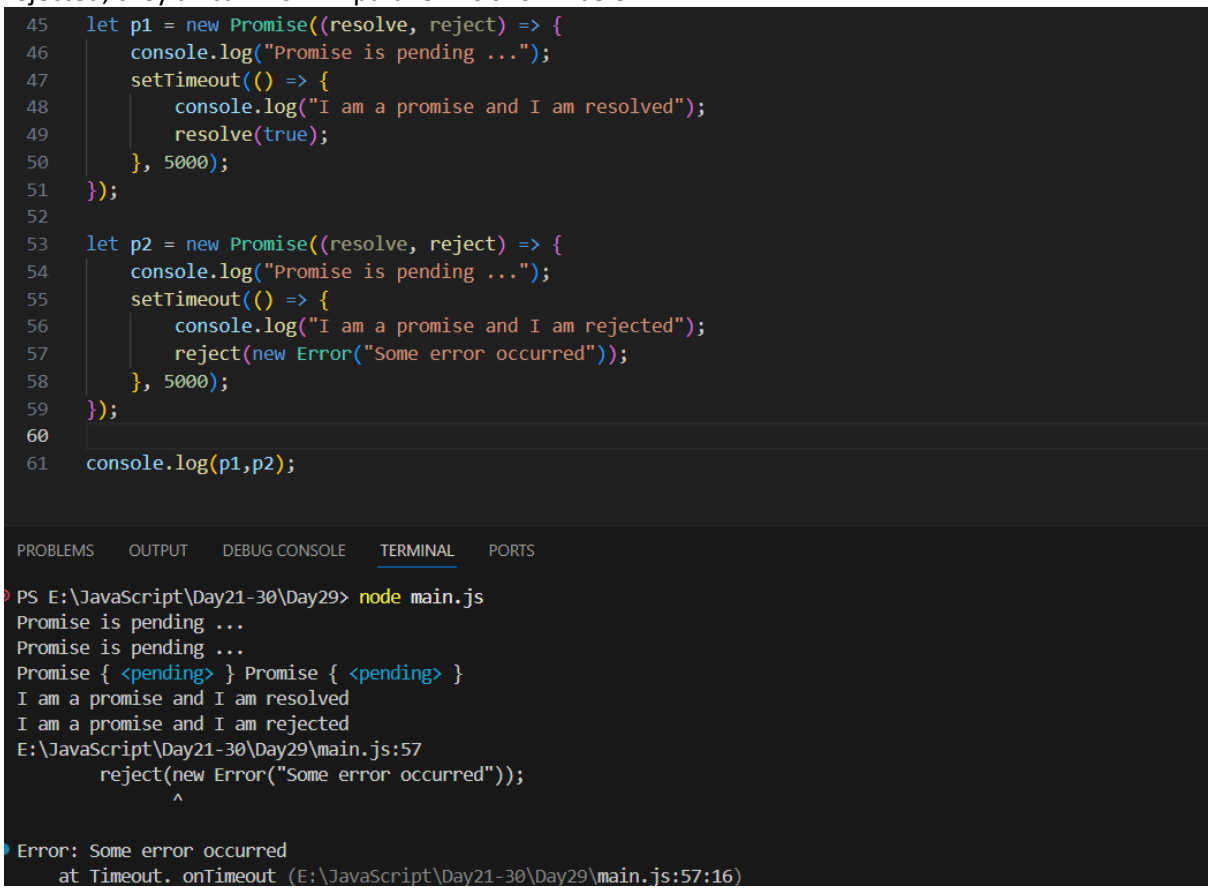
```
35  let promise = new Promise((resolve, reject) => {
36      console.log("Promise is pending ...");
37      setTimeout(() => {
38          console.log("I am a promise and I am rejected");
39          reject(new Error("Some error occurred"));
40      }, 5000);
41  });
42
43  console.log(promise); //it will show pending state as 5s are remaining
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS E:\JavaScript\Day21-30\Day29> node main.js
Promise is pending ...
Promise { <pending> }
```

After 5 seconds: error occurred as we wrote in the reject().

```
PS E:\JavaScript\Day21-30\Day29> node main.js
Promise is pending ...
Promise { <pending> }
I am a promise and I am rejected
E:\JavaScript\Day21-30\Day29\main.js:39
        reject(new Error("Some error occurred"));
               ^

Error: Some error occurred
    at Timeout._onTimeout (E:\JavaScript\Day21-30\Day29\main.js:39:16)
    at listOnTimeout (node:internal/timers:588:17)
    at process.processTimers (node:internal/timers:523:7)

Node.js v22.16.0
PS E:\JavaScript\Day21-30\Day29>
```

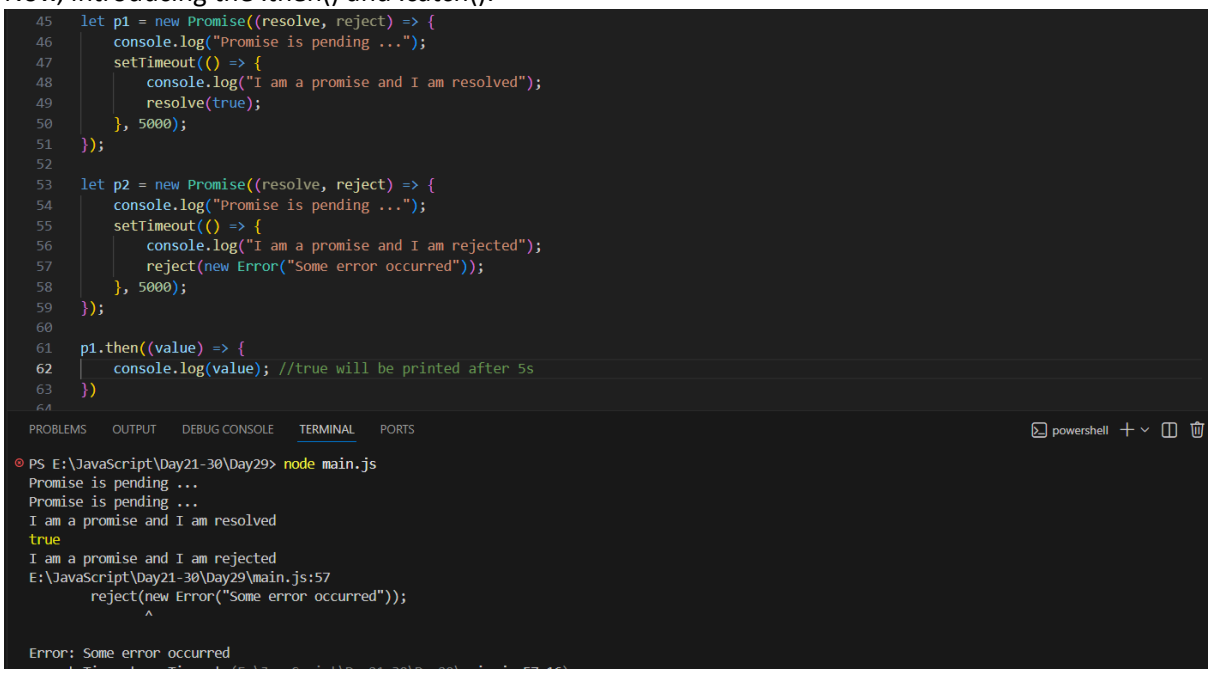
Also, promises executes parallelly. It means if we have multiple promises, either getting resolved or rejected, they all can work in parallel. As shown below:



```
let p1 = new Promise((resolve, reject) => {
    console.log("Promise is pending ...");
    setTimeout(() => {
        console.log("I am a promise and I am resolved");
        resolve(true);
    }, 5000);
});

let p2 = new Promise((resolve, reject) => {
    console.log("Promise is pending ...");
    setTimeout(() => {
        console.log("I am a promise and I am rejected");
        reject(new Error("Some error occurred"));
    }, 5000);
});

console.log(p1,p2);
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS E:\JavaScript\Day21-30\Day29> node main.js
Promise is pending ...
Promise is pending ...
Promise { <pending> } Promise { <pending> }
I am a promise and I am resolved
I am a promise and I am rejected
E:\JavaScript\Day21-30\Day29\main.js:57
        reject(new Error("Some error occurred"));
               ^

Error: Some error occurred
    at Timeout._onTimeout (E:\JavaScript\Day21-30\Day29\main.js:57:16)
```

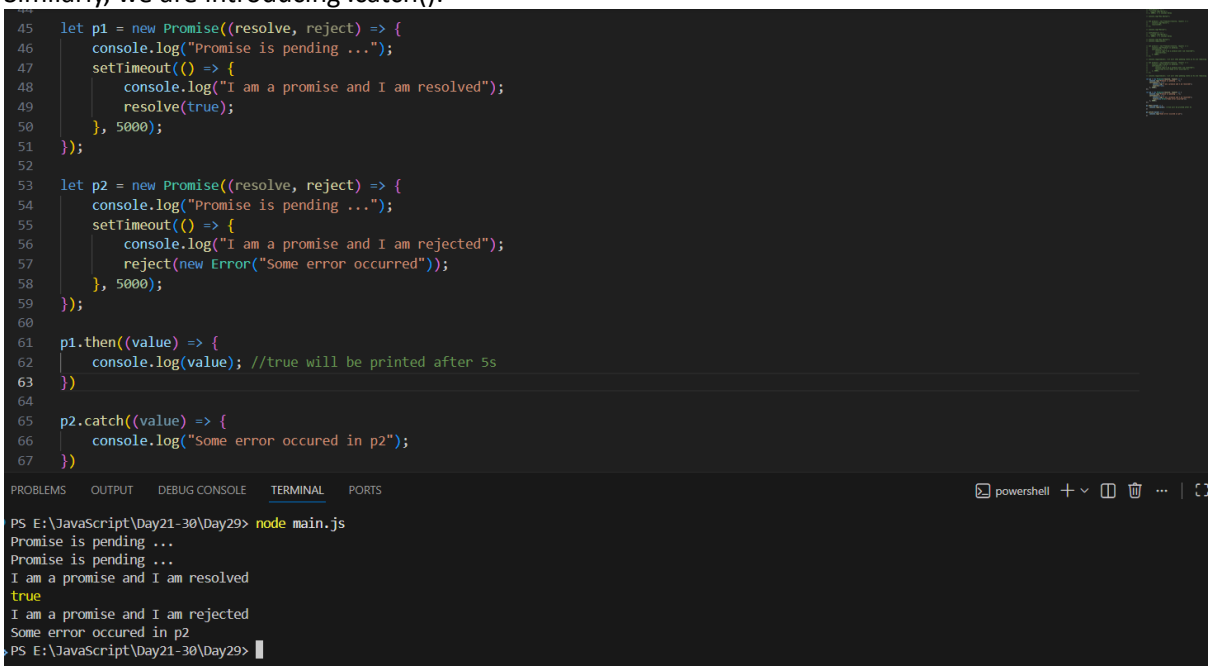
Now, introducing the .then() and .catch():



```
let p1 = new Promise((resolve, reject) => {
    console.log("Promise is pending ...");
    setTimeout(() => {
        console.log("I am a promise and I am resolved");
        resolve(true);
    }, 5000);
});

let p2 = new Promise((resolve, reject) => {
    console.log("Promise is pending ...");
    setTimeout(() => {
        console.log("I am a promise and I am rejected");
        reject(new Error("Some error occurred"));
    }, 5000);
});

p1.then((value) => {
    console.log(value); //true will be printed after 5s
})
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  powershell

PS E:\JavaScript\Day21-30\Day29> node main.js
Promise is pending ...
Promise is pending ...
I am a promise and I am resolved
true
I am a promise and I am rejected
E:\JavaScript\Day21-30\Day29\main.js:57
        reject(new Error("Some error occurred"));
               ^

Error: Some error occurred
```

Similarly, we are introducing .catch():



```
let p1 = new Promise((resolve, reject) => {
    console.log("Promise is pending ...");
    setTimeout(() => {
        console.log("I am a promise and I am resolved");
        resolve(true);
    }, 5000);
});

let p2 = new Promise((resolve, reject) => {
    console.log("Promise is pending ...");
    setTimeout(() => {
        console.log("I am a promise and I am rejected");
        reject(new Error("Some error occurred"));
    }, 5000);
});

p1.then((value) => {
    console.log(value); //true will be printed after 5s
})

p2.catch((value) => {
    console.log("Some error occured in p2");
})
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS        powershell

PS E:\JavaScript\Day21-30\Day29> node main.js
Promise is pending ...
Promise is pending ...
I am a promise and I am resolved
true
I am a promise and I am rejected
Some error occured in p2
PS E:\JavaScript\Day21-30\Day29>
```

--The End--